School Management System (SMS) blended with a **Learning Management System (LMS)**. It is a fantastic initiative because it bridges the communication gap between the school, the students, and the parents.

To get our brainstorm started, let's break this platform down into distinct user roles, core features, and a few extra ideas that could make your platform stand out.

---

1. User Roles & Dashboards

To keep the platform organized and secure, we should implement **Role-Based Access Control (RBAC)**. This means the platform will look different depending on who logs in:

- **School Admins/Management:** Have full control. They can create academic years, manage teacher/student accounts, upload official school documents, and send mass emails.
- **Teachers:** Have access to their specific classes. They can upload syllabuses, learning modules, assignments, and input student results.
- **Students:** Have a personalized dashboard to view their specific subjects, download study materials, check their results, and see school announcements.
- **Parents/Guardians:** Have a view-only dashboard linked to their child's profile to monitor academic progress, results, and receive school updates.

2. Core Modules

Based on your prompt, here is how we can structure the main functionalities:

- **Curriculum & Academic Management:**
    - A repository where teachers upload termly or yearly syllabuses.
    - A module section where students can download PDFs, slides, or reading materials for their registered subjects.

- **Result & Grading System:**
    - A secure portal where teachers input raw scores (tests, exams, assignments).
    - The system automatically calculates totals, grades, and GPAs based on the school's grading system.

    - Digital report cards that parents and students can view or download at the end of the term.

- **Document Vault:**
    - A centralized hub for school-related matters: academic calendars, school policies, tuition fee structures, and permission slips.

- **Automated Email Notification Engine:**
    - **Trigger-based emails:** "A new module has been uploaded for Mathematics," or "Term 1 Results have been published."
    - **School-wide broadcasts:** Newsletters, emergency closures, or upcoming PTA meetings.

3. Brainstorming Extras (Value Adds)

To make your platform even more robust, you might want to consider adding:

- **An Interactive Calendar:** A visual calendar showing exam dates, public holidays, and school events.
- **Attendance Tracking:** Teachers can mark daily attendance, and parents can receive an automatic email if their child is absent.
- **Mobile Responsiveness:** Many parents and students will access this via their smartphones, so the web platform must be optimized for mobile browsers (or built as a Progressive Web App).

Product Requirements Document (PRD)

Project Name: EduSync (Placeholder Name)

Document Version: 1.0

Date: February 19, 2026

1. Executive Summary

EduSync is a centralized, cloud-based educational management platform designed to streamline school operations, curriculum delivery, and result processing. It bridges the communication gap by providing dedicated, real-time access to school management, teachers, students, and parents, utilizing automated email notifications to keep all parties informed.

2. Objectives

- Digitize and secure student records, results, and school documents.
- Provide an easy-to-use portal for teachers to distribute syllabuses and learning modules.
- Increase transparency for parents regarding their child's academic performance.
- Automate school-wide and user-specific email communications.

---

3. User Roles & Permissions

User Role	Primary Goal	Key Permissions
School Admin	Manage the entire platform and school operations.	Create/manage users, set academic calendars, upload official documents, trigger school-wide emails.
Teacher	Manage assigned classes and assess students.	Upload syllabuses/modules, input raw scores, manage class-level resources.
Student	Access learning materials and track performance.	View/download modules, view grades, receive class updates.
Parent/Guardian	Monitor their child's academic progress.	View linked student's grades, download report cards, receive school updates.

---

4. Functional Requirements

These are the core features the system *must* have to function according to your vision.

- **Authentication & Authorization:** Secure login system with password recovery. Role-Based Access Control (RBAC) to ensure users only see what they are permitted to see.
- **Curriculum & Module Management:** An upload portal supporting PDFs, Word documents, and images. Materials must be categorized by Subject, Term, and Academic Year.
- **Result Processing System:** A secure interface for teachers to input test and exam scores. The system must automatically compute total scores, assign letter grades, and generate downloadable digital report cards.
- **Document Vault:** A centralized file manager for admins to upload school policies, fee structures, and newsletters.
- **Email Notification Engine:** Automated email triggers for account creation, password resets, new module uploads, published results, and custom admin broadcasts.

5. Non-Functional Requirements

These define how the system should perform and feel.

- **Security:** All user passwords must be encrypted. Student data must comply with standard data privacy regulations.
- **Mobile Responsiveness:** The user interface must automatically adapt to look great and function perfectly on smartphones, tablets, and desktops.
- **Reliability:** The platform should have automated daily database backups to prevent data loss.
- **Usability:** The dashboard must be highly intuitive, requiring little to no technical training for parents and students.

6. Out of Scope for MVP (Future Enhancements)

- Live video classes (Zoom/Google Meet integration).
- An internal messaging/chat system.
- Online tuition fee payment gateways.

DATABASE SCHEMA (THE BLUEPRINT)

To make this work, we need a relational database. This means our tables will "talk" to each other using IDs. Here is a simplified breakdown of the core tables:

- **Users Table (The Gatekeeper):**
  - UserID (Primary Key)
  - FullName
  - Email (Used for login and notifications)
  - PasswordHash (Encrypted for security)
  - Role (Admin, Teacher, Student, Parent)
- **Students Table (Extends User):**
  - StudentID
  - UserID (Linked to Users table)
  - ParentID (Linked to the Parent's UserID)
  - Class/GradeLevel
- **Subjects Table:**
  - SubjectID
  - SubjectName (e.g., Mathematics, Biology)
  - TeacherID (Linked to the Teacher's UserID)
- **Modules Table (Curriculum & Learning):**
  - ModuleID
  - SubjectID (Linked to Subjects)
  - Title (e.g., "Algebra Chapter 1")
  - DocumentURL (Link to the PDF/Doc stored in the cloud)
  - UploadDate
- **Results Table (Grading System):**
  - ResultID
  - StudentID (Linked to Students)
  - SubjectID (Linked to Subjects)
  - AcademicTerm (e.g., Term 1, 2026)
  - TestScore
  - ExamScore
  - TotalScore (Automatically computed)
- **School_Documents Table (Vault):**

- DocID
- Title (e.g., "2026 Academic Calendar")
- FileURL
- Visibility (Public, Internal Only)

USER INTERFACE (UI) SCREEN FLOW

1. Global Landing Page & Login

- **Screen:** Clean landing page with school branding.
- **Action:** User clicks "Portal Login", enters Email and Password.

2. The Admin Flow

- **Admin Dashboard:** High-level metrics (Total students, active teachers).
- **Manage Users Tab:** Add/Remove teachers, students, and parents. Link parent accounts to student accounts.
- **School Vault Tab:** Upload calendars and fee structures.
- **Broadcast Tab:** Type a message, select an audience (e.g., "All Parents"), and click "Send Email".

3. The Teacher Flow

- **Teacher Dashboard:** Shows a list of assigned classes/subjects.
- **Subject View:** Clicking a subject opens a workspace.
  - *Upload Module:* Select a file, add a title, hit "Upload". (This triggers an automatic email to students).
  - *Input Grades:* A spreadsheet-like view showing all students in that class to input test/exam scores.

4. The Student Flow

- **Student Dashboard:** Shows their registered subjects and recent school announcements.
- **My Subjects Tab:** Click on "Mathematics" -> See a list of downloadable PDFs and syllabus documents.
- **My Results Tab:** View current and past report cards.

5. The Parent Flow

- **Parent Dashboard:** View-only access. Shows their child's profile (or multiple children).

- **Academic Progress Tab:** View the child's grades and download the termly report card.
- **Notice Board Tab:** View official school documents and recent emails sent by the Admin.

THE TECH STACK (THE ENGINE)

For a secure, scalable platform that handles user roles, databases, and file uploads, you need a robust, modern tech stack. Here is a highly recommended setup for your Minimum Viable Product (MVP):

- **Frontend (What the user sees & interacts with):**
  - **React.js**: These are JavaScript frameworks that are perfect for building fast, interactive dashboards for your admins, teachers, students, and parents.
  - **Tailwind CSS**: For styling the platform quickly and ensuring it is perfectly mobile-responsive.
- **Backend (The brain of the operation):**
  - **Node.js (with Express)**: These handle the logic, authenticate users, and process data (like calculating student grades).
- **Database (Where the data lives):**
  - **PostgreSQL**: Since we designed a relational database schema earlier (where tables are linked by IDs), a SQL database is the most structured and reliable choice.
- **File Storage (For documents and modules):**
  - **Cloudinary**: You don't want to store heavy PDFs or images directly in your database. Instead, you store them in a secure cloud bucket and save the *link* to the file in your database.
- **Hosting:**
  - **Netlify** for the frontend, and **Render** for the backend and database.

---

2. Automated Email Notifications (The Communicator)

Building an automated email system means the platform does the heavy lifting without the Admin or Teacher having to manually draft emails every time.

To do this, we integrate a **Gmail API**. Here is how the trigger system flows:

The Workflow:

1. **The Event:** A teacher clicks "Upload" on a new Math syllabus.
2. **The Trigger:** The backend system detects that a new record was added to the Modules database table.
3. **The Audience Check:** The system looks up which students are enrolled in that specific Math class and grabs their registered email addresses (and their parents' emails, if configured).
4. **The Delivery:** The backend sends a pre-formatted template to the Gmail API with the dynamic data plugged in, which then safely delivers it to the users' inboxes.

Key Email Triggers for Your Platform:

- **Onboarding:** "Welcome to EduSync! Here are your login details." (Triggered when Admin creates a user).
- **Academic Updates:** "A new module (*Algebra Chapter 1*) has been uploaded by Mr Smith." (Triggered on module upload).
- **Result Notifications:** "Term 1 Results are now available on your dashboard." (Triggered when Admin clicks 'Publish Results').
- **Administrative Broadcasts:** Custom newsletters or alerts sent directly from the Admin's dashboard.

USER STORIES.

This is exactly how professional development teams break down a project so programmers know *why* they are building a feature, not just what it is. Then, we will look at a **Welcome Email Template** to see your automated communication engine in action.

---

1. Agile User Stories

In software development, we use the format: *"As a [User], I want to [Action], so that [Benefit/Reason]."* Here is a sample set for your developers:

For the School Admin:

- **As a School Admin,** I want to create teacher and student accounts, so that only authorized people can access the platform.

- **As a School Admin,** I want to upload official school documents to a centralized vault, so that parents and staff always have access to the latest policies and calendars.

For the Teacher:

- **As a Teacher,** I want to upload PDF modules and syllabuses to my assigned subject, so that my students can download their reading materials.
- **As a Teacher,** I want to input raw test and exam scores into the system, so that the platform can automatically calculate the final grades.

For the Student:

- **As a Student,** I want to log into my dashboard and see a list of my registered subjects, so that I can easily find my specific coursework.
- **As a Student,** I want to receive an automated email notification when a new module is uploaded, so that I never miss an assignment or reading.

For the Parent:

- **As a Parent,** I want my account linked to my child's profile, so that I can securely view their termly report cards.
- **As a Parent,** I want to receive school-wide broadcast emails, so that I am instantly informed about PTA meetings or school closures.

---

2. Automated Welcome Email Template (For Parents)

When an Admin creates a new Parent account, the system's email API will automatically plug in the specific details and send this welcome message.

Subject: Welcome to the EduSync Portal – Your Child's Academic Dashboard

Body:

Dear [Parent's First Name],

Welcome to [Name of School]'s new digital learning and management portal!

We are excited to launch this platform to bring you closer to [Student's First Name]'s academic journey. Through your secure parent dashboard, you will now be able to:

- View and download termly report cards.

- Access official school documents, calendars, and fee structures.
- Receive real-time notifications regarding school events and updates.

Your Login Credentials:

- **Portal Link:** [Insert Platform Web Address]
- **Username:** [Parent's Email Address]
- **Temporary Password:** [Auto-Generated Password]

For security purposes, the system will prompt you to change this temporary password the first time you log in.

If you have any questions or need technical assistance, please reply directly to this email or contact the school's admin office at [Support Phone Number].

Warm regards,

The Administration Team
[Name of School]

DURATION EXECUTION PLAN

3-Month Minimum Viable Product (MVP) is a realistic and highly effective approach. This allows you to build the core features, test them, and get a school using the platform as quickly as possible.
Here is a phased execution plan for building your educational platform:

Phase 1: Foundation & Architecture (Month 1)
Goal: Set up the scaffolding, secure the database, and build the Admin controls.

- **Week 1: Project Setup & Database Creation:** * Set up your frontend and backend environments (e.g., React and Node.js).
    - Create the database tables (Users, Students, Subjects, etc.) using the schema we mapped out.
- **Week 2: Authentication & Security:** * Build the secure login and logout system.
    - Implement Role-Based Access Control (RBAC) so the system knows the difference between an Admin, Teacher, Student, and Parent.
- **Week 3: The Admin Dashboard:** * Build the interface for Admins to create new users, assign roles, and set up the academic calendar.
- **Week 4: Document Vault Integration:** * Connect cloud storage (like Amazon S3) so Admins can upload official school documents securely.

Phase 2: Core Academic Features (Month 2)
Goal: Bring the classroom online by empowering teachers and students.

- **Week 5: Teacher & Subject Management:** * Build the Teacher Dashboard. Allow teachers to view their assigned subjects and class lists.
- **Week 6: Curriculum & Module Uploads:** * Create the feature for teachers to upload syllabuses and PDF modules.
    - Build the Student Dashboard so students can view and download these materials.
- **Week 7: The Grading System (Logic):** * Build the interface for teachers to input test and exam scores.
    - Write the backend math logic to automatically calculate total scores, percentages, and letter grades.

- **Week 8: Report Card Generation:** * Format the digital report card view for students to see their finalized results at the end of the term.

Phase 3: Communication, Polish, & Launch (Month 3)

Goal: Connect the parents, automate communications, and test everything.

- **Week 9: The Parent Portal:** * Build the view-only dashboard for parents, linking their accounts to their children's profiles so they can view results and school documents.
- **Week 10: Email Automation:** * Integrate your Email API (like SendGrid).
    - Set up the automated triggers (Welcome emails, new module alerts, result publication notices).
- **Week 11: Quality Assurance (QA) & Testing:** * Test every single button, upload form, and login screen on both desktop and mobile devices.
    - Fix any bugs or design glitches.
- **Week 12: Beta Launch & Onboarding:** * Deploy the platform to a live server.
    - Onboard your first test group (e.g., the school management team and a few teachers) to try it out in the real world.

RESTful API structure, which is the industry standard.

Here is the core map of the endpoints your developers will need to build, broken down by category.

1. Authentication & Account Access (/api/auth)

These endpoints handle getting users into the system securely.

Method	Endpoint	Description	Access Role
POST	/api/auth/login	Verifies email/password and returns a secure session token.	All Users
POST	/api/auth/logout	Ends the user's session and invalidates the token.	All Users
POST	/api/auth/forgot-password	Triggers the automated password reset email.	All Users
PUT	/api/auth/reset-password	Updates the password in the database.	All Users

2. User Management (/api/users)

These endpoints handle the creation and management of profiles.

Method	Endpoint	Description	Access Role
POST	/api/users	Creates a new user (triggers the Welcome Email).	Admin
GET	/api/users	Fetches a list of all users (can be filtered by role).	Admin
GET	/api/users/{id}	Fetches the specific profile details of a single user.	Admin, Self
PUT	/api/users/{id}/link-parent	Links a Parent's ID to a Student's profile.	Admin

3. Curriculum & Learning Modules (/api/modules)

Where teachers manage their classroom materials.

Method	Endpoint	Description	Access Role
POST	/api/modules	Uploads a new syllabus or document (triggers student email alert).	Teacher
GET	/api/modules/subject/{subjectId}	Fetches all uploaded materials for a specific subject.	Teacher, Student
DELETE	/api/modules/{id}	Removes a specific module or document.	Teacher, Admin

4. Results & Grading System (/api/results)
The endpoints are responsible for handling academic scores.

Method	Endpoint	Description	Access Role
POST	/api/results	Submits raw test/exam scores for a student.	Teacher
GET	/api/results/student/{studentId}	Fetches the calculated report card for a specific student.	Admin, Teacher, Student, Linked Parent
PUT	/api/results/{id}	Updates or corrects an existing score.	Teacher, Admin

5. Document Vault & Communications (/api/school)
Handling global school operations and broadcast emails.

Method	Endpoint	Description	Access Role
POST	/api/school/documents	Uploads official files (calendars, fee structures) to the vault.	Admin
GET	/api/school/documents	Fetches the list of available school documents.	All Users
POST	/api/school/broadcast	Triggers a mass email to a specific group (e.g., "All Parents").	Admin

admin@edusync.com
SuperSecretPassword123